

DOSW — Proyecto Pokédex



Plan de Trabajo — API REST por Capas

Java 21+ · Spring Boot 3.x · MapStruct · JWT · OAuth2 · Swagger · JaCoCo · SonarQube · JUnit

Tabla de Contenidos

1. Introducción y Contexto del Proyecto
2. Stack Tecnológico
3. Arquitectura por Capas — Estructura Detallada
4. Diagramas Requeridos y Cómo Generarlos (README)
5. Lunes — Setup, Config y Persistencia (Base)
6. Martes — Core: Entidades de Negocio y Servicios
7. Miércoles — Controller: DTOs, Mappers y Swagger
8. Jueves — Seguridad: JWT, OAuth2 y Autorización
9. Viernes — Pruebas, Cobertura y Análisis Estático
10. Lombok: Guía de Uso Correcto y Anti-Patrones
11. Problema N+1 y Cómo Evitarlo
12. Manejo de Errores y Excepciones
13. Material Complementario y Referencias

1. Introducción y Contexto del Proyecto

Este documento guía la construcción de la API REST de la Pokédex en 5 días (Lunes–Viernes). Está orientado a ingeniería de software real: arquitectura limpia, seguridad, pruebas y diagramas técnicos incluidos en el README.

1.1 Dominios del sistema

- Autenticación y perfiles de usuario (registro, login JWT, login Google/Gmail)
- Catálogo Pokémon: CRUD completo para ADMIN, consulta pública para todos
- Filtros avanzados: tipo, región, generación, stats, evoluciones, habilidades, movimientos
- Equipos Pokémon: cada usuario puede tener 1..N equipos con hasta 6 Pokémon
- Favoritos: guardar y destacar Pokémon de interés
- Estadísticas: tasa de elección en equipos, número de consultas, rankings
- Administración: gestión de usuarios por ADMIN

1.2 Roles del sistema

Rol	Permisos
GUEST	Ver listado y detalle de Pokémon (sin token)
TRAINER	Todo de GUEST + favoritos, equipos, perfil propio
ADMIN	Todo de TRAINER + CRUD Pokémon + administrar perfiles de usuarios

2. Stack Tecnológico

Tecnología	Versión / Propósito
Java 21 o Superior	Virtual Threads (Project Loom), Records nativos, Pattern Matching, Sealed Classes
Spring Boot 3.3.x	Framework principal. Requiere Java 21+; aprovecha features de Java
Spring Security 6.x	Autenticación, autorización, filtros JWT, OAuth2 Resource Server
JJWT (auth0 java-jwt)	Generación y validación de tokens JWT (HS256/RS256)
OAuth2 + Google	Login con Gmail — Spring Security OAuth2 Client
MapStruct 1.5.x	Mapeo en tiempo de compilación DTO↔Entity y Entity↔Model DAO
Lombok	Reducción de boilerplate. Ver Sección 10 para uso correcto
Spring Data JPA + PostgreSQL	Persistencia relacional principal. Driver postgresql
Spring Data MongoDB	Persistencia no relacional para estadísticas y logs de consultas

Springdoc OpenAPI 2.x	Documentación interactiva Swagger UI — http://localhost:8080/swagger-ui.html
JUnit 5 + Mockito	Pruebas unitarias y de integración
JaCoCo	Cobertura de código — meta mínima 70%
SonarLint	Análisis estático de calidad de código
Docker Compose	Levanta PostgreSQL + MongoDB localmente para desarrollo

Java 21 permite usar Records directamente como DTOs (inmutables, concisos), Sealed Classes para jerarquías de errores, y Virtual Threads para mejorar el rendimiento de operaciones I/O en Spring MVC.

3. Arquitectura por Capas — Estructura Detallada

El proyecto se organiza en 5 capas independientes. Cada capa solo conoce a la inmediatamente inferior. Ninguna capa puede saltarse otra.

Capa: controller

Interfaz HTTP. Recibe peticiones, valida DTOs, delega al Core, retorna respuestas.

- controller/ → Interfaces de controlador (@RequestMapping) + Implementaciones (@RestController)
- dto/request/ → Records Java: lo que recibe la API del cliente
- dto/response/ → Records Java: lo que retorna la API al cliente
- mapper/ → Interfaces MapStruct: convierte DTO ↔ Entidad de Negocio (Core)
- handler/ → GlobalExceptionHandler (@RestControllerAdvice)
- swagger/ → Configuración OpenAPI, anotaciones @Tag, @Operation, @ApiResponse

Capa: core

Corazón del sistema. Contiene toda la lógica de negocio, entidades y validaciones.

- service/interfaces/ → Contratos de servicio (interfaces)
- service/impl/ → Implementaciones de los servicios (@Service)
- model/ → Entidades de negocio (POJOs con Lombok, SIN anotaciones JPA)
- exception/ → Excepciones de negocio: ResourceNotFoundException, BusinessException, etc.
- validator/ → Validadores de negocio personalizados (@Component)
- util/ → Utilidades generales (DateUtils, StringUtils, etc.)

Capa: persistence

Acceso a datos. Mapea entre entidades de negocio (Core) y modelos DAO (base de datos).

- entity/relational/ → Entidades JPA (@Entity, @Table) — mapeadas a PostgreSQL
- entity/document/ → Documentos MongoDB (@Document) — estadísticas y logs
- repository/relational/ → Interfaces JpaRepository / JpaSpecificationExecutor
- repository/document/ → Interfaces MongoRepository
- mapper/ → Interfaces MapStruct: convierte Entidad de Negocio ↔ DAO/Documento

- adapter/ → Implementa los puertos de persistencia definidos en Core

Capa: config

Configuración transversal de la aplicación.

- AppConfig.java → Beans generales (@Bean, @Configuration)
- CorsConfig.java → Configuración CORS para frontend
- OpenApiConfig.java → Configuración global de Swagger/OpenAPI
- FlywayConfig.java → Configuración de migraciones (si se necesita override)
- AsyncConfig.java → Virtual Threads con Java 21 (Executors.newVirtualThreadPerTaskExecutor)

Capa: security

Seguridad transversal. Filtros, configuración Spring Security, JWT y OAuth2.

- SecurityConfig.java → SecurityFilterChain, reglas de autorización por endpoint
- JwtAuthFilter.java → OncePerRequestFilter — valida y procesa el token JWT
- JwtService.java → Genera, valida y extrae claims del token JWT
- OAuth2SuccessHandler.java → Maneja el callback de Google OAuth2, genera JWT propio
- UserDetailsServiceImpl.java → Carga el usuario desde BD para Spring Security

3.1 Estructura de paquetes completa

```
src/main/java/com/pokedex/
├── controller/                                ← CAPA CONTROLLER
│   ├── api/                                  (interfaces con @RequestMapping)
│   │   ├── PokemonApi.java
│   │   ├── AuthApi.java
│   │   ├── TeamApi.java
│   │   └── UserApi.java
│   ├── impl/                                (implementaciones @RestController)
│   │   ├── PokemonController.java
│   │   ├── AuthController.java
│   │   ├── TeamController.java
│   │   └── UserController.java
│   ├── dto/
│   │   ├── request/                          (Records: PokemonRequest, LoginRequest...)
│   │   └── response/                         (Records: PokemonResponse, TokenResponse...)
│   ├── mapper/                              (MapStruct: DTO ↔ Entidad de Negocio)
│   │   ├── PokemonDtoMapper.java
│   │   └── UserDtoMapper.java
│   ├── handler/
│   │   └── GlobalExceptionHandler.java
│   └── swagger/
│       └── OpenApiConfig.java
├── core/                                    ← CAPA CORE (lógica de negocio)
│   ├── service/
│   │   ├── interfaces/
│   │   │   ├── PokemonService.java
│   │   │   └── AuthService.java
```

```

├── TeamService.java
├── impl/
│   ├── PokemonServiceImpl.java
│   ├── AuthServiceImpl.java
│   └── TeamServiceImpl.java
├── model/                                (entidades de negocio - SIN JPA)
│   ├── Pokemon.java
│   ├── User.java
│   ├── Team.java
│   └── PokemonStats.java
├── exception/
│   ├── ResourceNotFoundException.java
│   ├── BusinessException.java
│   └── DuplicateResourceException.java
├── validator/
│   └── TeamValidator.java
├── util/
│   └── PaginationUtil.java
├── persistence/                          ← CAPA PERSISTENCE
│   ├── entity/
│   │   ├── relational/                  (entidades JPA → PostgreSQL)
│   │   │   ├── PokemonEntity.java
│   │   │   ├── UserEntity.java
│   │   │   └── TeamEntity.java
│   │   └── document/                    (documentos MongoDB)
│   │       ├── PokemonViewDocument.java
│   │       └── TeamStatsDocument.java
│   ├── repository/
│   │   ├── relational/
│   │   │   ├── PokemonJpaRepository.java
│   │   │   └── UserJpaRepository.java
│   │   └── document/
│   │       └── PokemonViewMongoRepository.java
│   ├── mapper/                          (MapStruct: Entidad de Negocio ↔ DAO)
│   │   ├── PokemonPersistenceMapper.java
│   │   └── UserPersistenceMapper.java
│   └── adapter/                          (implementa puertos de Core)
│       ├── PokemonPersistenceAdapter.java
│       └── UserPersistenceAdapter.java
├── config/                              ← CAPA CONFIG
│   ├── AppConfig.java
│   ├── CorsConfig.java
│   └── AsyncConfig.java                  (Virtual Threads - Java 21)
├── security/                             ← CAPA SECURITY
│   ├── SecurityConfig.java
│   ├── JwtAuthFilter.java
│   ├── JwtService.java
│   ├── OAuth2SuccessHandler.java
│   └── UserDetailsServiceImpl.java

```

3.2 Regla de dependencias entre capas

controller → core (solo interfaces de servicio) → persistence (a través de adapters). Las capas config y security son transversales. NUNCA: controller → persistence directamente, core → entity JPA.

4. Diagramas Requeridos y Cómo Incluirlos en el README

El README.md del proyecto debe incluir 5 diagramas realizados en Lucidchart/draw.io/Miro. A continuación se describe cada uno con su propósito.

4.1 Diagrama de Contexto (C4 - Nivel 1)

Muestra el sistema como una caja negra y sus actores e integraciones externas.

4.2 Diagrama de Componentes General (C4 - Nivel 2)

Muestra las 5 capas internas y sus responsabilidades principales.

4.3 Diagrama de Componentes Específico (flujo Pokemon)

Muestra el flujo completo de una petición GET /api/v1/pokemon/{id} a través de todas las capas.

4.4 Diagrama de Clases

Muestra las clases principales, sus atributos y relaciones. Foco en los objetos de la capa Core

4.5 Diagrama Entidad-Relación (PostgreSQL) y Documentos (MongoDB)

Muestra las relaciones entre los DAOS de persistencia relacional, el otro diagrama muestra las relaciones y generación de los documentos implementados para persistencia no relacional.

Incluye todos estos diagramas con su respectiva explicación técnica en el archivo README.md en la raíz del proyecto.

5. Lunes — Setup, Configuración y Persistencia (Base)

Objetivo del día: Proyecto corriendo con Docker, bases de datos conectadas, entidades JPA y documentos MongoDB creados, migraciones Flyway funcionando.

5.1 Crear proyecto con Spring Initializr

Ve a <https://start.spring.io> y configura:

- Project: Maven · Language: Java · Spring Boot: 3.3.x · Java: 21 o superior
- Dependencias: Spring Web, Spring Data JPA, Spring Data MongoDB, Spring Security, Lombok, Validation, PostgreSQL Driver

5.2 docker-compose.yml

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: pokedex_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    ports: ['5432:5432']
    volumes: [postgres_data:/var/lib/postgresql/data]

  mongo:
    image: mongo:7-jammy
    ports: ['27017:27017']
    volumes: [mongo_data:/data/db]

volumes:
  postgres_data:
  mongo_data:
```

5.3 application.yml

```
spring:
  application:
    name: nombre-de-la-pokedex
  datasource:
    url: jdbc:postgresql://localhost:5432/pokedex_db
    username: postgres
    password: password
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: validate      # Flyway controla el esquema
      show-sql: false
      open-in-view: false    # IMPORTANTE: evitar lazy loading en la vista
  data:
    mongodb:
      uri: mongodb://localhost:27017/pokedex_stats
  flyway:
    enabled: true
    locations: classpath:db/migration

server:
  port: 8080
  servlet:
    context-path: /api

app:
  jwt:
    secret: ${JWT_SECRET}      # Variable de entorno - nunca hardcoded
    expiration-ms: 86400000    # 24 horas
```


5.4 Entidad JPA — persistence/entity/relational/

Las entidades JPA viven SOLO en la capa persistence. La capa core NO las conoce.

Ejemplo: (Varía dependiendo de sus implementaciones)

```
@Entity
@Table(name = "pokemon",
      indexes = { @Index(name = "idx_pokemon_number", columnList = "national_number")
    })
@Getter // Lombok: genera SOLO getters (no setters en entidad JPA)
@NoArgsConstructor(access = AccessLevel.PROTECTED) // JPA necesita constructor sin
args
@AllArgsConstructor(access = AccessLevel.PRIVATE) // Solo Builder puede llamar al
constructor completo
@Builder // Patrón Builder para construcción segura
public class PokemonEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "national_number", nullable = false, unique = true)
    private Integer nationalNumber;

    @Column(nullable = false, length = 100)
    private String name;

    // CORRECTO: FetchType.LAZY para colecciones – se carga solo si se pide
    explícitamente
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "pokemon_type",
        joinColumns = @JoinColumn(name = "pokemon_id"),
        inverseJoinColumns = @JoinColumn(name = "type_id")
    )
    private List<TypeEntity> types = new ArrayList<>();

    // CORRECTO: OneToOne con LAZY (evita carga innecesaria de stats)
    @OneToOne(mappedBy = "pokemon", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    private PokemonStatsEntity stats;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "region_id")
    private RegionEntity region;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}
```

5.5 Documento MongoDB — persistence/entity/document/

Los documentos viven SOLO en la capa persistence. La capa core NO las conoce.

Ejemplo: (Varía dependiendo de sus implementaciones)

```
@Document(collection = "pokemon_views")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PokemonViewDocument {

    @Id
    private String id;

    @Field("pokemon_id")
    private Long pokemonId;

    @Field("pokemon_name")
    private String pokemonName;

    @Field("view_count")
    private Long viewCount = 0L;

    @Field("last_viewed")
    private LocalDateTime lastViewed;
}
```

6. Martes — Core: Entidades de Negocio y Servicios

Objetivo del día: Crear los modelos de negocio (POJOs puros), interfaces de servicio, implementaciones e inyección de dependencias a través de adapters de persistencia.

6.1 Modelo de negocio (core/model) — sin anotaciones JPA

Los modelos del core son POJOs limpios. No tienen `@Entity`, no dependen de ningún framework de persistencia. Esto permite cambiar la BD sin tocar la lógica de negocio.

```
@Value          // Lombok: clase inmutable — todos los campos final, solo getters, no
setters
@Builder(toBuilder = true) // toBuilder permite crear copias modificadas
public class Pokemon {
    Long id;
    Integer nationalNumber;
    String name;
    String description;
    String imageUrl;
    List<String> types; // Solo los nombres, no referencias a otras entidades
    String region;
}
```

```
    Integer generation;
    Boolean hasMega;
    PokemonStats stats;
}

@Value
@Builder
public class PokemonStats {
    Integer hp;
    Integer attack;
    Integer defense;
    Integer specialAttack;
    Integer specialDefense;
    Integer speed;

    public Integer getTotal() {
        return hp + attack + defense + specialAttack + specialDefense + speed;
    }
}
```

6.2 Excepciones de negocio (core/exception)

```
// Clase base
public class BusinessException extends RuntimeException {
    private final String errorCode;
    public BusinessException(String message, String errorCode) {
        super(message);
        this.errorCode = errorCode;
    }
    public String getErrorCode() { return errorCode; }
}

// Excepción específica - usa Java 21 Record para compactness
public class ResourceNotFoundException extends BusinessException {
    public ResourceNotFoundException(String resource, String field, Object value) {
        super(resource + " con " + field + "=" + value + " no encontrado", "NOT_FOUND");
    }
}

public class DuplicateResourceException extends BusinessException {
    public DuplicateResourceException(String resource, String field, Object value) {
        super(resource + " con " + field + "=" + value + " ya existe", "DUPLICATE");
    }
}
```

6.3 Interface de servicio + implementación

```
public interface PokemonService {
    Page<Pokemon> findAll(Pageable pageable);
    Pokemon findById(Long id);
}
```

```

    Pokemon findByNationalNumber(Integer number);
    Pokemon create(Pokemon pokemon);
    Pokemon update(Long id, Pokemon pokemon);
    void delete(Long id);
    List<Pokemon> filterByCriteria(PokemonFilterCriteria criteria);
}

@Service
@RequiredArgsConstructor // Lombok: constructor con todos los campos final → inyección
                        // de dependencias
@Slf4j                // Lombok: logger =
LoggerFactory.getLogger(PokemonServiceImpl.class)
public class PokemonServiceImpl implements PokemonService {

    private final PokemonPersistencePort pokemonPort; // Puerto (interface) — no la
implementación

    @Override
    public Pokemon findById(Long id) {
        log.debug("Buscando Pokemon con id: {}", id);
        return pokemonPort.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Pokemon", "id", id));
    }

    @Override
    @Transactional
    public Pokemon create(Pokemon pokemon) {
        if (pokemonPort.existsByNationalNumber(pokemon.getNationalNumber())) {
            throw new DuplicateResourceException("Pokemon", "nationalNumber",
pokemon.getNationalNumber());
        }
        return pokemonPort.save(pokemon);
    }
}

```

6.4 Puerto de persistencia (interface en core)

El Core define lo que necesita (el puerto), la capa persistence lo implementa (el adapter). Esto es el principio de Inversión de Dependencias.

```

public interface PokemonPersistencePort {
    Optional<Pokemon> findById(Long id);
    Page<Pokemon> findAll(Pageable pageable);
    boolean existsByNationalNumber(Integer number);
    Pokemon save(Pokemon pokemon);
    void deleteById(Long id);
}

@Component
@RequiredArgsConstructor
public class PokemonPersistenceAdapter implements PokemonPersistencePort {

```

```
private final PokemonJpaRepository repository;
private final PokemonPersistenceMapper mapper;

@Override
public Optional<Pokemon> findById(Long id) {
    return repository.findByIdWithTypesAndStats(id) // @EntityGraph
        .map(mapper::toDomain);
}

@Override
public Pokemon save(Pokemon pokemon) {
    PokemonEntity entity = mapper.toEntity(pokemon);
    return mapper.toDomain(repository.save(entity));
}
}
```

7. Miércoles — Controller: DTOs, Mappers y Swagger

Objetivo del día: DTOs con Java Records, interfaces de controlador con anotaciones Swagger, implementaciones, mappers MapStruct DTO↔Core y GlobalExceptionHandler.

7.1 DTOs con Java Records (controller/dto)

```
public record PokemonRequest(
    @NotNull(message = "El número nacional es obligatorio")
    @Min(value = 1, message = "El número debe ser mayor a 0")
    Integer nationalNumber,

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede exceder 100 caracteres")
    String name,

    @NotBlank
    String imageUrl,

    @NotEmpty(message = "Debe tener al menos un tipo")
    @Size(max = 2, message = "Un Pokémon puede tener máximo 2 tipos")
    List<String> types,

    @NotNull Long regionId,
    @NotNull @Min(1) Integer generation
) {}

public record PokemonResponse(
    Long id,
```

```

    Integer nationalNumber,
    String name,
    String imageUrl,
    List<String> types,
    String region,
    Integer generation,
    Boolean hasMega,
    PokemonStatsResponse stats
) {}

public record PokemonStatsResponse(
    Integer hp, Integer attack, Integer defense,
    Integer specialAttack, Integer specialDefense,
    Integer speed, Integer total
) {}

```

7.2 Interfaz del controlador con Swagger (controller/api)

```

@Tag(name = "Pokemon", description = "Gestión del catálogo de Pokémon")
@RequestMapping("/v1/pokemon")
public interface PokemonApi {

    @Operation(summary = "Listar todos los Pokémon",
        description = "Retorna lista paginada. Acceso público.")
    @ApiResponseResponses({
        @ApiResponse(responseCode = "200", description = "Lista obtenida",
            content = @Content(schema = @Schema(implementation =
                PokemonResponse.class))),
        @ApiResponse(responseCode = "400", description = "Parámetros inválidos")
    })
    @GetMapping
    ResponseEntity<Page<PokemonResponse>> findAll(
        @PageableDefault(size = 20, sort = "nationalNumber") Pageable pageable);

    @Operation(summary = "Obtener Pokémon por ID")
    @ApiResponse(responseCode = "404", description = "Pokémon no encontrado")
    @GetMapping("/{id}")
    ResponseEntity<PokemonResponse> findById(@PathVariable Long id);

    @Operation(summary = "Crear Pokémon", description = "Solo ADMIN")
    @SecurityRequirement(name = "Bearer Authentication")
    @PostMapping
    ResponseEntity<PokemonResponse> create(@Valid @RequestBody PokemonRequest request);

    @Operation(summary = "Actualizar Pokémon", description = "Solo ADMIN")
    @SecurityRequirement(name = "Bearer Authentication")
    @PutMapping("/{id}")
    ResponseEntity<PokemonResponse> update(@PathVariable Long id,
        @Valid @RequestBody PokemonRequest request);

    @Operation(summary = "Eliminar Pokémon", description = "Solo ADMIN")
    @SecurityRequirement(name = "Bearer Authentication")
    @DeleteMapping("/{id}")

```

```

    ResponseEntity<Void> delete(@PathVariable Long id);
}

```

7.3 Implementación del controlador (controller/impl)

```

@RestController
@RequiredArgsConstructor
public class PokemonController implements PokemonApi {

    private final PokemonService pokemonService;
    private final PokemonDtoMapper mapper;

    @Override
    public ResponseEntity<Page<PokemonResponse>> findAll(Pageable pageable) {
        return
            ResponseEntity.ok(pokemonService.findAll(pageable).map(mapper::toResponse));
    }

    @Override
    public ResponseEntity<PokemonResponse> findById(Long id) {
        return ResponseEntity.ok(mapper.toResponse(pokemonService.findById(id)));
    }

    @Override
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<PokemonResponse> create(PokemonRequest request) {
        Pokemon created = pokemonService.create(mapper.toDomain(request));
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}").buildAndExpand(created.getId()).toUri();
        return ResponseEntity.created(location).body(mapper.toResponse(created));
    }
}

```

7.4 Mapper MapStruct — controller/mapper/

```

@Mapper(componentModel = "spring")
public interface PokemonDtoMapper {

    // Pokemon (core) → PokemonResponse (DTO)
    // MapStruct mapea automáticamente campos con el mismo nombre
    @Mapping(source = "stats.specialAttack", target = "stats.specialAttack")
    PokemonResponse toResponse(Pokemon pokemon);

    // PokemonRequest (DTO) → Pokemon (core)
    @Mapping(target = "id", ignore = true) // id lo asigna la BD
    @Mapping(target = "hasMega", constant = "false") // valor por defecto
    Pokemon toDomain(PokemonRequest request);

    List<PokemonResponse> toResponseList(List<Pokemon> pokemons);
}

```

```

}

@Mapper(componentModel = "spring")
public interface PokemonPersistenceMapper {

    // PokemonEntity (JPA) → Pokemon (core model)
    @Mapping(source = "region.name", target = "region")
    @Mapping(expression =
"java(entity.getTypes().stream().map(TypeEntity::getName).toList())",
        target = "types")
    Pokemon toDomain(PokemonEntity entity);

    // Pokemon (core model) → PokemonEntity (JPA)
    @Mapping(target = "region", ignore = true) // se resuelve en el adapter
    @Mapping(target = "types", ignore = true) // se resuelve en el adapter
    PokemonEntity toEntity(Pokemon pokemon);
}

```

8. Jueves — Seguridad: JWT, OAuth2 y Autorización

Objetivo del día: Implementar SecurityConfig, JwtAuthFilter, JwtService, OAuth2 con Google, y proteger todos los endpoints según roles.

8.1 SecurityConfig

```

// security/SecurityConfig.java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // Activa @PreAuthorize en métodos
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final OAuth2SuccessHandler oAuth2SuccessHandler;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable) // API REST stateless
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/v1/auth/**").permitAll()
                .requestMatchers(GET, "/v1/pokemon/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(o -> o.successHandler(oAuth2SuccessHandler))

```



```

        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

8.2 JwtService

```

// security/JwtService.java
@Service
@Slf4j
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration-ms}")
    private long expirationMs;

    public String generateToken(UserDetails userDetails) {
        return Jwts.builder()
            .subject(userDetails.getUsername())
            .claim("roles", userDetails.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
    }
}

```

8.3 Tabla de endpoints y seguridad

Método	Ruta (/api/v1...)	Rol	Descripción
--------	-------------------	-----	-------------

POST	/auth/register	Público	Registro de usuario
POST	/auth/login	Público	Login → retorna JWT
GET	/auth/oauth2/google	Público	Login con Gmail
GET	/pokemon	Público	Listar Pokémon (paginado)
GET	/pokemon/{id}	Público	Detalle de un Pokémon
GET	/pokemon/filter	Público	Filtrar por tipo, región, gen...
POST	/pokemon	ADMIN	Crear Pokémon
PUT	/pokemon/{id}	ADMIN	Actualizar Pokémon
DELETE	/pokemon/{id}	ADMIN	Eliminar Pokémon
GET	/teams	USER	Ver mis equipos
POST	/teams	USER	Crear equipo Pokémon
GET	/favorites	USER	Ver mis favoritos
POST	/favorites/{pokemonId}	USER	Agregar a favoritos
GET	/stats	USER+ADMIN	Estadísticas generales
GET	/admin/users	ADMIN	Administrar usuarios

9. Viernes — Pruebas, Cobertura y Análisis Estático

Objetivo del día: Pruebas unitarias para Service y Controller, configurar JaCoCo (meta ≥70%), ejecutar Checkstyle, verificar checklist final.

9.1 Prueba unitaria del servicio

```
@ExtendWith(MockitoExtension.class)
class PokemonServiceImplTest {

    @Mock private PokemonPersistencePort pokemonPort;
    @InjectMocks private PokemonServiceImpl service;

    // Fixture reusable
    private Pokemon pikachu;

    @BeforeEach
    void setUp() {
        pikachu = Pokemon.builder()
            .id(1L).nationalNumber(25).name("Pikachu")
            .types(List.of("Electric")).region("Kanto")
            .generation(1).hasMega(false)
            .stats(PokemonStats.builder().hp(35).attack(55).defense(40)
                .specialAttack(50).specialDefense(50).speed(90).build())
    }
}
```

```

        .build();
    }

    @Test
    @DisplayName("findById: debe retornar el Pokemon cuando existe")
    void findById_whenExists_returnsP() {
        when(pokemonPort.findById(1L)).thenReturn(Optional.of(pikachu));
        Pokemon result = service.findById(1L);
        assertThat(result).isNotNull();
        assertThat(result.getName()).isEqualTo("Pikachu");
        verify(pokemonPort).findById(1L);
    }

    @Test
    @DisplayName("findById: debe lanzar ResourceNotFoundException cuando no existe")
    void findById_whenNotFound_throws() {
        when(pokemonPort.findById(99L)).thenReturn(Optional.empty());
        assertThrows(ResourceNotFoundException.class, () -> service.findById(99L));
    }

    @Test
    @DisplayName("create: debe lanzar DuplicateResourceException si el número ya existe")
    void create_whenDuplicate_throws() {
        when(pokemonPort.existsByNationalNumber(25)).thenReturn(true);
        assertThrows(DuplicateResourceException.class, () -> service.create(pikachu));
        verify(pokemonPort, never()).save(any());
    }
}

```

9.2 Prueba del controlador con @WebMvcTest

```

@WebMvcTest(PokemonController.class)
@AutoConfigureMockMvc(addFilters = false) // Sin filtros de seguridad en pruebas unitarias
class PokemonControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @MockBean private PokemonService pokemonService;
    @MockBean private PokemonDtoMapper mapper;

    @Test
    void findById_returns200() throws Exception {
        Pokemon pokemon = Pokemon.builder().id(1L).name("Pikachu").build();
        PokemonResponse response = new PokemonResponse(1L, 25, "Pikachu",
            null, List.of("Electric"), "Kanto", 1, false, null);

        when(pokemonService.findById(1L)).thenReturn(pokemon);
        when(mapper.toResponse(pokemon)).thenReturn(response);

        mockMvc.perform(get("/v1/pokemon/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Pikachu"))
            .andExpect(jsonPath("$.types[0]").value("Electric"));
    }
}

```

```

    }

    @Test
    void create_withInvalidBody_returns400() throws Exception {
        PokemonRequest invalid = new PokemonRequest(null, "", null, List.of(), null,
null);
        mockMvc.perform(post("/v1/pokemon")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(invalid)))
            .andExpect(status().isBadRequest());
    }
}

```

9.3 JaCoCo — configuración pom.xml

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <configuration>
    <excludes>
      <exclude>**/*Config.*</exclude>
      <exclude>**/*Application.*</exclude>
      <exclude>**/dto/**</exclude>          <!-- Records: sin lógica -->
      <exclude>**/entity/**</exclude>       <!-- Entidades JPA -->
      <exclude>**/document/**</exclude>     <!-- Documentos Mongo -->
    </excludes>
  </configuration>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution>
      <id>report</id><phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
    <execution>
      <id>check</id><goals><goal>check</goal></goals>
      <configuration><rules><rule>
        <element>BUNDLE</element>
        <limits><limit>
          <counter>LINE</counter>
          <value>COVEREDRATIO</value>
          <minimum>0.70</minimum>  <!-- Mínimo 70% -->
        </limit></limits>
      </rule></rules></configuration>
    </execution>
  </executions>
</plugin>
# Ver reporte: mvn test → target/site/jacoco/index.html

```

9.4 Checklist final de entrega

✓	Ítem	Verificación
[]	Proyecto compila sin errores	mvn clean compile

[]	Todos los tests pasan	<code>mvn test</code>
[]	Cobertura $\geq 70\%$ reportada	<code>target/site/jacoco/index.html</code>
[]	Análisis estático sin errores críticos	<code>mvn checkstyle:check</code>
[]	Swagger UI accesible y con todos los endpoints	<code>/swagger-ui.html</code>
[]	Login JWT retorna token válido	<code>POST /v1/auth/login</code>
[]	OAuth2 Google redirige correctamente	<code>GET /v1/auth/oauth2/google</code>
[]	Endpoints ADMIN bloqueados para USER	Probar con token USER
[]	Endpoints públicos sin token funcionan	Sin header Authorization
[]	Paginación funciona en listado	<code>GET /pokemon?page=0&size=10</code>
[]	Filtros funcionan (tipo, región, gen)	<code>GET /pokemon/filter?type=fire</code>
[]	N+1 resuelto con <code>@EntityGraph</code>	Ver logs SQL con <code>show-sql: true</code>
[]	Error 404 retorna formato <code>ApiError</code>	<code>GET /pokemon/99999</code>
[]	JWT_SECRET en variable de entorno	No hardcoded en yaml
[]	5 diagramas en README.md	Ver sección 4 de este documento
[]	MongoDB guarda estadísticas de vistas	Ver colección <code>pokemon_views</code>

10. Lombok: Guía de Uso Correcto y Anti-Patrones

Lombok reduce el boilerplate, pero usado incorrectamente genera bugs difíciles de detectar. Esta sección muestra cuándo y cómo usar cada anotación.

10.1 Anotaciones y cuándo usarlas

Anotación	Úsala en	Descripción y precaución
<code>@Getter</code> / <code>@Setter</code>	Entidades JPA, DTOs mutables	Prefiere <code>@Getter</code> solo en entidades JPA. Nunca <code>@Setter</code> en entidades — usa builders o métodos de negocio.
<code>@Data</code>	DTOs simples, clases de utilidad	Genera <code>@Getter</code> + <code>@Setter</code> + <code>@ToString</code> + <code>@EqualsAndHashCode</code> + <code>@RequiredArgsConstructor</code> . EVITAR en entidades JPA (rompe equals/hashCode con Hibernate).
<code>@Value</code>	Modelos de negocio (core/model)	Clase inmutable: todos los campos final, solo getters. Ideal para modelos de dominio. Combinar con <code>@Builder</code> .
<code>@Builder</code>	Entidades JPA, modelos de negocio	Patrón Builder. En entidades JPA combinar con <code>@NoArgsConstructor(PROTECTED)</code> + <code>@AllArgsConstructor(PRIVATE)</code> .
<code>@RequiredArgsConstructor</code>	Services, Controllers, Adapters	Constructor con todos los campos final. Esencial para inyección de dependencias en Spring (NO necesitas <code>@Autowired</code>).

@NoArgsConstructor	Entidades JPA, documentos MongoDB	JPA requiere constructor sin args. Usar access = AccessLevel.PROTECTED para no exponerlo innecesariamente.
@Slf4j	Services, Filters, Handlers	Genera: private static final Logger log = LoggerFactory.getLogger(ClaseActual.class). Usa log.debug/info/warn/error.
@EqualsAndHashCode	Con cuidado en entidades JPA	En entidades JPA incluir solo el ID: @EqualsAndHashCode(onlyExplicitlyIncluded = true) con @EqualsAndHashCode.Include en el id.

10.2 Ejemplo correcto para entidad JPA

```

@Entity
@Table(name = "user")
@Getter // Solo getters - nunca @Setter en entidad
@Builder
@NoArgsConstructor(access = AccessLevel.PROTECTED) // JPA lo necesita, pero protegido
@AllArgsConstructor(access = AccessLevel.PRIVATE) // Solo Builder puede construir
@EqualsAndHashCode(onlyExplicitlyIncluded = true) // Evita comparar colecciones lazy
public class UserEntity {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @EqualsAndHashCode.Include // Solo el ID en equals/hashCode
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    private Role role;

    // Método de negocio en lugar de setter
    public UserEntity withRole(Role newRole) {
        return this.toBuilder().role(newRole).build();
    }
}

```

10.3 Ejemplo correcto para modelo de negocio (core)

```

@Value // Inmutable: todos final, solo getters
@Builder(toBuilder = true) // toBuilder permite crear copias con campos modificados
public class Pokemon {
    Long id;
    String name;
    List<String> types;

    // Uso de toBuilder para crear una copia con hasMega=true
    // Pokemon megaPokemon = pikachu.toBuilder().hasMega(true).build();
}

// INCORRECTO - Nunca @Data en modelos de dominio

```

```
// @Data ← genera setters → modelo mutable → bugs difíciles de detectar
// public class Pokemon { ... }
```

10.4 Ejemplo correcto para servicios y componentes Spring

```
// CORRECTO — Servicio con inyección de dependencias vía constructor
@Service
@RequiredArgsConstructor // Spring inyecta automáticamente los campos final
@Slf4j // Habilita log.info(...), log.debug(...), etc.
public class PokemonServiceImpl implements PokemonService {

    private final PokemonPersistencePort pokemonPort; // final = inmutable = seguro
    private final TeamValidator teamValidator;

    @Override
    public Pokemon create(Pokemon pokemon) {
        log.info("Creando Pokemon: {}", pokemon.getName());
        teamValidator.validatePokemon(pokemon);
        return pokemonPort.save(pokemon);
    }
}

// INCORRECTO — @Autowired en campo (no usar con Lombok)
// @Autowired
// private PokemonPersistencePort pokemonPort; ← campo no final, dificulta pruebas
```

11. Problema N+1 en JPA y Cómo Evitarlo

El problema N+1 ocurre cuando cargar 1 lista de entidades genera N consultas adicionales para cargar sus relaciones. Es uno de los problemas de rendimiento más comunes con JPA y DEBE evitarse.

11.1 ¿Cuándo ocurre N+1?

Ejemplo: cargar 151 Pokémon genera 1 consulta para la lista + 151 consultas para cargar los tipos de cada uno = 152 consultas en total. Con FetchType.EAGER el problema es automático; con LAZY ocurre cuando accedes a la colección fuera de la transacción.

```
// PROBLEMA: este código genera N+1
List<PokemonEntity> pokemons = repository.findAll(); // 1 query
pokemons.forEach(p -> p.getTypes().size());          // N queries (1 por Pokémon)

// En los logs verías:
// SELECT * FROM pokemon                               ← 1 query
// SELECT * FROM type JOIN pokemon_type WHERE pokemon_id=1 ← N queries
// SELECT * FROM type JOIN pokemon_type WHERE pokemon_id=2
// ... (una por cada pokemon)
```

11.2 Solución 1: @EntityGraph (recomendada para consultas específicas)

```
public interface PokemonJpaRepository extends JpaRepository<PokemonEntity, Long> {

    // @EntityGraph hace JOIN FETCH en una sola consulta
    @EntityGraph(attributePaths = {"types", "stats", "region"})
    Optional<PokemonEntity> findByIdWithTypesAndStats(@Param("id") Long id);

    // Para listados paginados — solo carga lo necesario
    @EntityGraph(attributePaths = {"types"})
    @Query("SELECT p FROM PokemonEntity p")
    Page<PokemonEntity> findAllWithTypes(Pageable pageable);

    // Para filtros — carga tipos y región en una sola query
    @EntityGraph(attributePaths = {"types", "region"})
    List<PokemonEntity> findByRegion_Name(String regionName);
}
```

11.3 Solución 2: Projections (solo los campos que necesitas)

```
// Para el listado general, NO necesitas todos los campos
// Usa una projection para cargar solo lo necesario
public interface PokemonSummary {
    Long getId();
    Integer getNationalNumber();
    String getName();
    String getImageUrl();
}

public interface PokemonJpaRepository extends JpaRepository<PokemonEntity, Long> {
    // Spring Data genera: SELECT id, national_number, name, image_url FROM pokemon
    Page<PokemonSummary> findAllProjectedBy(Pageable pageable);
}

// Solo usa el findAllWithTypes (@EntityGraph) cuando realmente necesitas los tipos
// Usa la projection para el listado básico — mucho más eficiente
```

11.4 Regla general para evitar N+1

Caso de uso	Estrategia recomendada
Detalle de 1 entidad	@EntityGraph con TODOS los atributos que necesitas
Listado paginado simple	Projection — solo campos escalares (sin colecciones)
Listado con tipos/relaciones	@EntityGraph con solo los atributos necesarios
Búsqueda/filtro	@EntityGraph + @Query con condiciones WHERE
Estadísticas/conteos	@Query con COUNT, GROUP BY — no cargar entidades

NUNCA usar

FetchType.EAGER en colecciones (@ManyToMany, @OneToMany) — genera joins masivos en toda consulta

Verifica N+1: activa temporalmente `spring.jpa.show-sql: true` y revisa cuántas queries genera una petición al endpoint de listado. Si ves más de 2 queries para una lista, hay un problema N+1.

12. Manejo de Errores y Excepciones

12.1 Formato estándar de error (ApiError)

```
public record ApiError(
    int status,
    String errorCode,
    String message,
    String path,
    LocalDateTime timestamp,
    List<FieldError> fieldErrors // Para errores de validación
) {
    public record FieldError(String field, String message) {}
}

// Ejemplo de respuesta 404:
// { "status": 404, "errorCode": "NOT_FOUND",
//   "message": "Pokemon con id=99 no encontrado",
//   "path": "/api/v1/pokemon/99",
//   "timestamp": "2025-01-01T10:00:00",
//   "fieldErrors": [] }
```

12.2 GlobalExceptionHandler

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(
        ResourceNotFoundException ex, HttpServletRequest req) {
        log.warn("Recurso no encontrado: {}", ex.getMessage());
        return status(404).body(buildError(404, ex.getErrorCode(),
            ex.getMessage(), req.getRequestURI(), List.of()));
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ApiError> handleDuplicate(
        DuplicateResourceException ex, HttpServletRequest req) {
        return status(409).body(buildError(409, ex.getErrorCode(),
            ex.getMessage(), req.getRequestURI(), List.of()));
    }
}
```

```
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiError> handleValidation(
    MethodArgumentNotValidException ex, HttpServletRequest req) {
    List<ApiError.FieldError> errors = ex.getBindingResult().getFieldErrors()
        .stream().map(e -> new ApiError.FieldError(e.getField(),
            e.getDefaultMessage())).toList();
    return status(400).body(buildError(400, "VALIDATION_ERROR",
        "Error de validación en los datos de entrada",
        req.getRequestURI(), errors));
}

private ApiError buildError(int status, String code, String msg,
    String path, List<ApiError.FieldError> errors) {
    return new ApiError(status, code, msg, path, LocalDateTime.now(), errors);
}
}
```

13. Material Complementario y Referencias

13.1 Fundamentos: IoC, SOLID y Spring

- [Principios SOLID VI: Inversión de Dependencias \(DIP\)](#)
- [Inversión de Control \(IoC\) explicado](#)
- [Inversión de Control — Oscar Blancarte](#)
- [Intro a Project Lombok \(Baeldung\)](#)
- [Spring Boot Tutorial completo \(Baeldung\)](#)
- [Getting Started con Spring Boot \(spring.io\)](#)
- [Spring @Component y jerarquía de anotaciones](#)
- [Patrón MVC explicado \(Código Facilito\)](#)
- [Records en Java — KeepCoding](#)

13.2 API REST, Swagger y Versionamiento

- [REST e idempotencia \(Adictos al Trabajo\)](#)
- [MapStruct — Guía completa \(Baeldung\)](#)
- [Swagger UI oficial](#)
- [Swagger Editor online](#)
- [Manejo de errores REST — IBM Docs](#)
- [Exception handling en Spring REST \(Baeldung\)](#)
- [Versionamiento de APIs REST — mejores prácticas](#)
- [Versionado semántico \(SemVer\)](#)

13.3 Persistencia: JPA, MongoDB y Relaciones

-
- [JPA con Spring Boot — guía completa](#)
 - [Spring Data JPA — Getting Started \(spring.io\)](#)
 - [Relaciones JPA: OneToOne, OneToMany, ManyToMany](#)
 - [MongoDB Atlas Tutorial oficial](#)
 - [MongoDB: Embeber vs Referenciar documentos](#)
 - [AWS: ¿Qué es NoSQL?](#)
 - [Modelado de bases NoSQL](#)
 - [Propiedades ACID — Datacamp](#)

13.4 Seguridad

- [SSO con Spring Security y OAuth2 \(Baeldung\)](#)
- [Spring Boot OAuth2 Tutorial \(spring.io\)](#)
- [Spring Security OAuth2 — documentación oficial](#)
- [JWT Auth con Spring Boot 3 + MySQL](#)
- [Autenticación con Spring Security y JWT](#)
- [Spring Security JWT Resource Server — docs](#)
- [OWASP Top 10 — 2025](#)
- [Clickjacking — Fortinet](#)
- [CSRF — Fortinet](#)
- [XSS — Mozilla MDN](#)

13.5 Diagramas

- [Diagramas de Secuencia UML \(Lucidchart\)](#)
- [Diagrama de Secuencia UML \(Miro\)](#)
- [Diagrama Entidad-Relación \(Lucidchart\)](#)

13.6 Videos — por día de la semana

Lunes — Setup y Spring Boot:

- [Spring Boot Tutorial completo — Java Brains](#)
- [Spring Boot 3 tutorial paso a paso](#)
- [Spring Boot + PostgreSQL completo](#)
- [Spring Data JPA — relaciones avanzadas](#)
- [JPA OneToMany y ManyToMany con Spring](#)
- [Spring Boot + MongoDB tutorial](#)

Martes — Core y Servicios:

- [MapStruct en Spring Boot](#)
- [Lombok tutorial práctico](#)
- [API REST con Spring Boot \(tutorial completo\)](#)

Miércoles — Controllers y Swagger:

- [Documentar APIs con Swagger y Springdoc](#)
- [Spring Boot REST — controladores y DTOs](#)

Jueves — Seguridad:

- [Spring Security 6 + JWT — tutorial completo](#)
- [Spring Boot + JWT + MySQL \(completo\)](#)
- [Spring Security OAuth2 con Google](#)
- [JWT Auth con Spring Security](#)
- [Spring Boot 3 Security completo](#)

Viernes — Pruebas:

- [JUnit 5 y Mockito — tutorial completo](#)
- [Testing Spring Boot — playlist completa](#)
- [Spring Boot Test tutorial](#)
- [Pruebas de integración con Spring Boot](#)

14. Organización del Trabajo con Jira

Jira es la herramienta de gestión de proyectos ágiles. Para este proyecto individual se configura un tablero Scrum con épicas por capa, historias de usuario por funcionalidad y tareas técnicas por día. La organización en Jira NO es opcional: demuestra planificación y trazabilidad como lo hace un equipo real.

14.1 Configuración inicial en Jira

1. Crear cuenta gratuita en <https://www.atlassian.com/es/software/jira>
2. Nuevo proyecto → Scrum → Nombre: Pokedex API Clave del proyecto: POK
3. Crear 1 Sprint — duración: 1 semana (Lunes a Viernes)
4. Columnas del tablero: To Do | In Progress | In Review | Done
5. Habilitar campos: Story Points, Priority, Due Date, Component/s (una por capa)

14.2 Jerarquía Epica → Historia → Tarea

Nivel	Tipo en Jira	Descripción y ejemplo
Epica	Epic	Agrupar funcionalidades grandes.
Historia	Story	Funcionalidad desde la perspectiva del usuario. Estimada en Story Points (1, 2, 3, 5, 8). Ej: Como admin puedo crear un Pokemon
Tarea técnica	Task / Sub-task	Trabajo técnico específico. Se descompone de la Story. Ej: Crear entidad PokemonEntity con Lombok y JPA

14.4 Backlog completo — Historias por epica

Copia cada issue como Story en Jira con la épica correspondiente. Los Story Points siguen la escala de Fibonacci: 1, 2, 3, 5, 8.

14.6 Convención de commits vinculada a Jira

Cada commit debe referenciar el issue. Esto crea trazabilidad automática entre código y backlog:

```
# Formato: <tipo>(POK-XXX): descripcion breve en minusculas

feat(POK-001): crear estructura de paquetes por 5 capas
feat(POK-005): crear PokemonEntity con Lombok correcto y JPA
feat(POK-006): agregar @EntityGraph a PokemonJpaRepository para evitar N+1
feat(POK-010): definir modelo de negocio Pokemon inmutable con @Value @Builder
feat(POK-018): implementar PokemonController con interfaz PokemonApi y Swagger
feat(POK-026): configurar SecurityConfig con roles USER y ADMIN
test(POK-028): agregar pruebas unitarias para PokemonServiceImpl
fix(POK-031): corregir exclusiones JaCoCo para entidades y DTOs
docs(POK-033): agregar diagrama de contexto C4 al README

# Tipos: feat | fix | test | docs | refactor | config
```

14.7 Flujo del tablero — Como mover issues

6. Inicio del día: arrastra a In Progress los issues que haras hoy
7. Al terminar el código: mueve a In Review (revisión propia o del profesor)
8. Al pasar pruebas y Checkstyle: mueve a Done — aplica el DoD completo
9. Si encuentras un bug: crea Issue tipo Bug, enlazarlo con 'relates to' a un issue padre
10. Final del día: verifica que el Burndown del Sprint refleje el avance real

14.8 Definition of Done (DoD)

Un issue pasa a Done SOLO cuando cumple TODOS estos criterios simultáneamente.

	Criterio — el issue esta Done cuando...
[]	El código compila sin errores: mvn clean compile
[]	Pruebas unitarias escritas y pasando para el issue (cuando aplica)
[]	La cobertura del módulo no bajo respecto al issue anterior
[]	Sin errores de Checkstyle en los archivos del issue
[]	El commit sigue el formato feat(POK-XXX): descripcion
[]	El endpoint aparece documentado en Swagger si el issue genera uno
[]	El issue esta en columna Done en el tablero Jira
[]	No se rompieron pruebas existentes: mvn test pasa completamente

¡Mucho éxito con tu Pokédex API! 🎮

Código limpio · Capas bien separadas · Pruebas sólidas · Seguridad de verdad